

Study Guide — Module 4 Evaluation - Attempt 1

Contents

1. Machine Learning Foundations	1
2. Simple Linear Regression	2
3. Multiple Linear Regression	3
4. Regression Evaluation Metrics	4
5. Logistic Regression & the Sigmoid Function	6
6. Decision Trees for Classification	8
7. Confusion Matrix Fundamentals	11
8. Classification Metrics: Accuracy, Precision, Recall, F1 Score	12
9. Choosing Between Regression and Classification Models	15
10. Feature Selection & ML Pipelines	17
11. Related Algorithms	19
Coverage checklist	19

Batch: IITREICT-AIML-2604 · **Program:** Artificial Intelligence and Machine Learning Certification Program

This guide compiles the key concepts you need for your upcoming evaluation. Work through each section, focus on the worked examples and definitions, and use the key terms to self-test.

1. Machine Learning Foundations

Supervised vs Unsupervised Learning

- **Supervised Learning:** Both input (x) and output (y) variables are available as pairs. The algorithm learns the mapping from x to y . Labeled data contains both features and a target variable, allowing the model to learn from known outcomes.
- **Unsupervised Learning:** Only input (x) data is available, with no labeled output (y). Algorithms try to find patterns or clusters without supervision. Unlabeled data contains only features and requires the model to find patterns independently.

Regression vs Classification

Machine learning problems are categorized based on whether the output variable y is continuous (regression) or discrete (classification).

- **Regression Problems:** Output variable is continuous — it can take an infinite number of numerical values within a range.
 - Examples: salary prediction, house price prediction, temperature forecasting, solar energy forecasting, stock price forecasting, delivery-time prediction.

- Equation form: $Y = MX + C$ (or in ML notation, $y = w_0 + w_1 \cdot x$).
- **Classification Problems:** Output variable is discrete — it has fixed, countable values.
 - **Binary Classification:** exactly two possible outcome classes (e.g. spam / not spam, loan approval yes/no, pass/fail, disease/no disease, fraudulent/not fraudulent).
 - **Multi-class Classification:** more than two possible outcome classes (e.g. fruit type — apple/banana/strawberry; customer net worth — high/medium/low; Iris species — Setosa/Versicolor/Virginica; sentiment — positive/neutral/negative).
 - **Important:** the number of input **features** used has no relationship with whether a problem is binary or multi-class — only the number of **outcome classes** matters.

Regression and classification are fundamentally different problem types. Even if two models are built from the same data source, a regression model cannot be directly compared to a classification model — this is like comparing the nutritional value of an orange to an apple. Always choose evaluation metrics that match the problem type, and never average metrics across the two types (e.g. averaging R^2 with F1 Score produces a meaningless result).

Key terms: supervised learning, unsupervised learning, labeled/unlabeled data, regression, classification, binary classification, multi-class classification.

2. Simple Linear Regression

- **Goal:** model the linear relationship between one input variable (x) and a continuous output variable (y).
- **Equation:**
 - Traditional math form: $y = mx + c$
 - ML notation: $y = w_0 + w_1 \cdot x$
 - w_0 (intercept): the predicted value of y when $x = 0$ (e.g. base salary with zero experience).
 - w_1 (coefficient/slope): the effect of x on y (e.g. increment in salary per year of experience).
- **Assumption:** Y depends linearly on x , though the method can tolerate some deviation from perfect linearity.
- **Examples of linear relationships:** housing price vs house size, salary vs years of experience, exam marks vs hours studied.
- **Why use it:** easy and fast to train, highly interpretable, can be used as a baseline model, and sometimes outperforms complex models when the underlying data is genuinely linear.

Training & Prediction Workflow

1. Load the data (e.g. experience and salary).

2. Plot the data to observe linearity.
3. Split the data into training and test sets (e.g. 80% train / 20% test) using `train_test_split`.
4. Create a `LinearRegression` model object.
5. Train the model with `.fit()` on the training data.
6. Retrieve the model's coefficients and intercept.
7. Predict on the test set using `.predict()`.
8. Calculate evaluation metrics (MAE, MSE, RMSE, R^2) on the **test set**.
9. Use the model to predict outputs for new input values.

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

model = LinearRegression()
model.fit(X_train, y_train)           # X must be a 2D array, even for a single
feature
predictions = model.predict(X_test)
model.coef_                          # weight(s) for each feature
model.intercept_                    # bias/intercept term
```

- **Implementation notes:** when predicting on multiple new samples with a single feature, reshape the input appropriately — e.g. `new_x.reshape(-1, 1)` — to keep it a 2D array. The number of input features at prediction time must match the number of features the model was trained on, **exactly**.

Key terms: best-fit line, intercept, slope/coefficient, `train_test_split`, `fit()`, `predict()`, `reshape()`.

3. Multiple Linear Regression

Multiple linear regression extends simple linear regression to multiple input features/predictors.

- **Equation:** $y = w_0 + w_1x_1 + w_2x_2 + w_3x_3 + \dots + w_nx_n$
 - Each feature contributes independently to the output based on its own weight (w).
 - w_0 is the baseline predicted value when all features are zero.
 - Each coefficient w_i represents how much the predicted output changes per unit change in its corresponding feature, **holding the other features constant**.
 - Coefficients are learned from and depend on the training data — they are not fixed constants; a negative coefficient indicates an inverse relationship with the output (not “no effect”).
- **Example scenarios:** predicting salary from experience + education + communication skills; house price from size + bedrooms + amenities + location; student performance from attendance + study hours + previous GPA.

- **Feature importance:** including key features improves accuracy, but irrelevant features may not contribute positively and may be discarded experimentally. Some features can be **redundant** — e.g. age can indirectly capture experience.

- **Preparing data (professional approach):**

```
X = df.drop('target_column', axis=1)
y = df['target_column']
```

- **Workflow:** load/visualize data → build X (feature matrix) and y (target vector) → `train_test_split` (e.g. 80/20 or 70/30) → `.fit(X_train, y_train)` → inspect intercept/coefficients → `.predict(X_test)` → evaluate with MAE/MSE/RMSE/R².

Worked Interpretation: Salary Prediction Example

- Using a salary-prediction dataset with experience, education, and age as features, visualizing each feature against salary individually (before fitting the model) showed a positive relationship for all three — a useful sanity check before trusting the fitted coefficients.
- After training, **experience had the largest positive weight** — around **INR 4,000 of additional salary per extra year of experience**, holding education and age constant — while education carried a moderate positive weight and age contributed a smaller effect (and can also be partly **redundant** with experience, since more years of age often coincide with more years of experience).
- **Manual prediction walkthrough:** once `.fit()` produces the intercept w_0 and the per-feature weights w_1 , w_2 , w_3 , a new prediction is reproduced by hand as $\text{predicted_salary} = w_0 + w_1 \cdot \text{experience} + w_2 \cdot \text{education} + w_3 \cdot \text{age}$ — substituting a candidate's own experience/education/age values and the model's learned weights gives exactly the same number that `.predict()` returns, which confirms what the fitted equation is actually computing.

Limitations of Linear Regression

- Assumes a linear relationship between features and output — can fail for complex or non-linear data.
- Sensitive to outliers, which can distort the fitting line dramatically; outliers should be detected and handled (removed or treated separately) before modeling.

Key terms: multiple linear regression, coefficients, intercept, redundant features, feature matrix.

4. Regression Evaluation Metrics

Errors are defined as $\text{Error} = \text{Actual Value} - \text{Predicted Value}$. Absolute error ignores the sign; squared error eliminates negative values by squaring the difference.

Mean Absolute Error (MAE)

Average of the absolute differences between actual and predicted values. Lower is better.

Mean Squared Error (MSE)

$MSE = (1/N) * \sum (y_i - \hat{y}_i)^2$ — the average of the squared errors.

- Always non-negative; closer to zero is better.
- Squaring the residual emphasizes (penalizes) larger errors more than small ones.
- Differentiable and computationally convenient for optimization during model training.
- Highly **sensitive to outliers** since squaring amplifies large deviations.

Worked Example: Manual MSE/RMSE Verification

Using a house-size vs. house-price dataset with a fitted `LinearRegression` model, the manual-verification workflow is:

1. For each row, compute the **residual**: `residual = actual_price - predicted_price`.
2. **Square** each residual to remove the sign.
3. **Average** the squared residuals — this manual MSE should match `mean_squared_error(y_test, y_pred)` from scikit-learn exactly; computing it by hand once and checking it against the library function is a good way to confirm you understand what the metric is actually doing.
4. Take the square root of that MSE to get **RMSE**, which is interpretable directly in the same units as the target (e.g. Rupees, for a price target).
5. Plot actual vs. predicted points, and draw each row's residual as a **dashed line** connecting the actual value to the predicted value — this visualizes exactly which points the model over- or under-predicts, and by how much.

Root Mean Squared Error (RMSE)

Square root of MSE — brings the error back into the same unit as the output variable, which makes it easier to interpret and report to stakeholders. Lower is better.

R-squared (R²) — Coefficient of Determination

$R^2 = 1 - [\sum (y_i - \hat{y}_i)^2 / \sum (y_i - \bar{y})^2]$, where $\bar{y}$ is the mean of the actual values.

- R^2 compares the model's performance against a **baseline model that simply predicts the mean** of the target for every input.
- Measures the proportion of variance in the output that is explained by the model.
- Range: typically 0 to 1 (can be negative if the model is worse than the baseline).
 - $R^2 = 1$: perfect model.
 - $R^2 = 0.5$: commonly used as a rough midpoint marker of a middling model (what counts as “good” varies by domain/context).
 - $R^2 = 0$: the model performs exactly as well as the mean-predicting baseline — no better, no worse. (A model that performs genuinely worse than the baseline has a **negative** R^2 .)
- R^2 should ordinarily be compared only across models evaluated under the **same dataset, split, target definition, and evaluation procedure** — its value is normalized to a 0–1-ish range, but that normalization does not make scores from

unrelated datasets meaningfully comparable, since R^2 depends on the target's own variance and the specific evaluation population.

MSE vs R^2 — Complementary Metrics

Aspect	MSE	R^2
Measures	Average squared prediction error	Proportion of variance explained vs. baseline (mean)
Range	0 to ∞	$-\infty$ to 1
Interpretation	Lower is better	Closer to 1 is better
Sensitivity to outliers	High	Also sensitive to outliers
Best use	Training/testing error magnitude	Goodness of fit relative to a baseline
Cross-dataset comparison	Not advisable without the same data	Also not advisable without the same data

Both metrics should always be reported together — they provide different, complementary insights. Report held-out validation/test performance to assess how well the model generalizes to unseen data, while also inspecting training-set metrics when diagnosing fit and tuning model complexity (e.g. comparing training vs. test accuracy across different tree depths — see Section 6). Remove or address outliers before training, since both MSE and R^2 are sensitive to them.

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np
```

```
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
```

Key terms: MAE, MSE, RMSE, R^2 , baseline (mean-predicting) model, residuals.

5. Logistic Regression & the Sigmoid Function

Logistic regression is a **classification** algorithm despite the word “regression” in its name — it predicts the probability that an observation belongs to a particular class, making it suitable for categorical outcomes.

- Linear regression can produce invalid values (e.g. 1.3 or 2.9) for a binary outcome, which have no meaningful interpretation. Logistic regression instead constrains predictions to a **probability range of 0 to 1**.

The Sigmoid (Logistic) Function

$\sigma(z) = 1 / (1 + e^{(-z)})$, where $z = w_0 + w_1 \cdot x_1 + \dots + w_n \cdot x_n$ (a linear combination of inputs and weights).

- Sigmoid outputs values strictly between 0 and 1 — an S-shaped curve, smoothly transitioning from 0 (large negative z) to 1 (large positive z), passing through 0.5 exactly when $z = 0$.
- The output is interpreted as the probability of belonging to the positive class.

Decision Boundary & Threshold

- The **decision boundary** is the point where the predicted probability equals the threshold.
- **Default threshold:** 0.5 — a probability at or above the threshold predicts the positive class (“class 1”); below it predicts the negative class (“class 0”).
- The threshold can be adjusted based on application requirements (e.g. a more conservative threshold in medical diagnosis). Raising the threshold makes the model more conservative about predicting the positive class (fewer positive predictions); lowering it makes the model predict positive more readily.

Worked Example: Study Hours → Pass/Fail

Using a toy dataset of hours studied vs. pass/fail outcome:

- Fitting LogisticRegression on hours studied against pass/fail produced a decision boundary at **approximately 4 hours of study** — the point where the predicted probability of passing crosses 0.5.
- **Manual computation:** for a given number of study hours x , compute $z = w_0 + w_1 \cdot x$ from the model’s fitted intercept and coefficient, then apply $\sigma(z) = 1 / (1 + e^{(-z)})$ to get the predicted probability of passing; if that probability is ≥ 0.5 the model predicts “pass”, otherwise “fail” — this reproduces exactly what `.predict()` / `.predict_proba()` return.
- Visualizing the fitted **sigmoid curve** (S-shaped, bounded between 0 and 1) alongside a straight **linear-regression line** fit to the same hours-vs-outcome data makes the case for logistic regression concrete: the straight line can predict values below 0 or above 1 for very low or very high study hours, which have no valid probability interpretation, while the sigmoid stays bounded and interpretable across the full range of inputs.

Workflow & Key Functions

```
from sklearn.linear_model import LogisticRegression
```

```
clf = LogisticRegression()  
clf.fit(X_train, y_train)  
clf.predict(X_test)           # final predicted class labels, using the  
threshold  
clf.predict_proba(X_test)     # predicted probability for each class  
(probabilities sum to 1)
```

- For binary classification with a single feature: `X = df[['feature_name']]` (double brackets keep it a 2D array), `y = df['target_name']`.
- For multi-class / multiple features: `X = df.drop('target_column', axis=1)`, `y = df['target_column']`, then `train_test_split`.
- Classes can be represented as text (e.g. “Pass”/“Fail”) or numbers (0, 1, 2, ...).
- Loading a predefined dataset (e.g. Iris, a classic multi-class example):

```
from sklearn.datasets import load_iris
iris = load_iris()
X = iris.data           # feature values
y = iris.target         # class labels
iris.feature_names
iris.target_names
```

Key terms: sigmoid function, decision boundary, threshold, `predict()`, `predict_proba()`.

6. Decision Trees for Classification

A decision tree is a **rule-based** algorithm — it makes predictions by asking a hierarchical sequence of nested if-else conditions about features, unlike learning-based algorithms (logistic regression, etc.) which learn weight parameters through an optimization process.

Terminology

- **Root Node:** the top-level node where the tree begins; contains the first decision question.
- **Internal Node:** any node with child nodes, representing an intermediate decision.
- **Branch:** the connection between nodes, representing the outcome of a decision.
- **Leaf Node:** a terminal node with no children — represents the final classification output (e.g. pass/fail, approved/rejected).

Real-world analogy: deciding whether to carry an umbrella — “Is it raining?” → yes: carry umbrella; no → “Is it cloudy?” → yes: carry umbrella; no → skip the umbrella.

Gini Impurity Index

$Gini = 1 - \sum (\text{probability of class } i)^2$

- Measures impurity at a node and guides which feature/threshold to split on.
- **Pure node:** contains instances of only one class (Gini = 0).
- **Impure node:** contains mixed-class instances (Gini > 0).
- Process: compute Gini before the split (parent node), compute Gini for each child branch after the split, then compute the **weighted Gini**: $(n_{\text{left}}/n_{\text{total}}) \times Gini_{\text{left}} + (n_{\text{right}}/n_{\text{total}}) \times Gini_{\text{right}}$. The best split is the one that maximizes the reduction in impurity (i.e. gives the lowest weighted Gini).

- **Entropy** is an alternative split criterion in scikit-learn (`criterion='entropy'`) that serves the same purpose — measuring how well a candidate split separates the classes. Gini is the **default** criterion.

Building the Tree

1. Start at the root node, evaluate all available features/conditions.
2. Select the best split (lowest weighted Gini / highest impurity reduction).
3. Create branches based on the split condition.
4. Recurse on each impure branch using the remaining features.
5. Stop when reaching pure nodes or a maximum depth limit.

At each node, the tree reevaluates all eligible features and candidate thresholds against the data that reached that node — a feature is not permanently “used up” after one split. In particular, a continuous or otherwise reusable feature can be selected again at a descendant node with a **different threshold**, if that gives the best impurity reduction for the subset of data at that node.

Worked Example: Pass/Fail Decision Tree

Using a student pass/fail dataset:

- **Root split:** `hours_of_study >= 4`.
 - **Right branch** (`hours ≥ 4`): every student in this branch passes — a **pure node** (Gini = 0); no further split is needed here.
 - **Left branch** (`hours < 4`): a mix of pass and fail outcomes — an **impure node** (Gini > 0), so the tree recurses on this branch using the remaining features.
- **Secondary split** (on the impure left branch): `attended_recovery_class?`
 - **No:** every student in this sub-branch fails — pure node (Gini = 0).
 - **Yes:** still a mix of pass/fail outcomes — remains impure; with more depth or more features the tree would keep splitting here.
- This shows the general rule in action: a branch stops splitting once it becomes pure (or a stopping condition like `max_depth` is hit); an impure branch keeps recursing through whatever features remain.

Overfitting & Underfitting

- **Overfitting:** the tree grows too deep, trying to classify every single training instance correctly. This memorizes specific training instances rather than general patterns, producing excellent training accuracy but poor generalization to new data.
- **Underfitting:** the tree is too shallow, stopping prematurely, so it fails to classify the majority of training instances correctly (often from setting `max_depth` too low).
- **Controlling depth:** the `max_depth` parameter limits how deep the tree can grow. There is no universal optimal value — the standard approach is to test several depths (e.g. 2, 3, 4, 5, 10), compare training vs. test accuracy for each, and choose the depth that balances accuracy with generalization.

```

from sklearn.tree import DecisionTreeClassifier, plot_tree

clf = DecisionTreeClassifier(criterion='gini', max_depth=2, random_state=42)
clf.fit(X_train, y_train)
clf.predict(X_test)
clf.score(X_test, y_test)          # accuracy
plot_tree(clf, feature_names=..., class_names=...) # visualize the tree

```

Advantages & Limitations

- **Advantages:** highly interpretable/explainable (each decision path can be traced), visualizable, works with raw (unscaled) data, naturally handles multi-class problems, gives a clear feature-importance signal, non-parametric (no assumptions about the data distribution).
- **Limitations:** prone to overfitting with complex trees; can be unstable (small data changes can produce very different trees); biased toward features with more levels/splits.

Decision Trees vs Logistic Regression

Aspect	Decision Tree	Logistic Regression
Mechanism	Rule-based nested if-else conditions	Learns weight parameters via optimization
Parameters	No linear weight vector; learns split features/thresholds and per-leaf class statistics via greedy criterion optimization	$w_0, w_1, \dots, w_n$
Optimization	Greedy, per-node impurity optimization (not a global weight-fitting process)	Requires an iterative optimization process
Complexity	Simple to describe	More complex to describe
Output	Class label; <code>predict_proba()</code> also returns the class proportions observed at the reached leaf	Probability estimate + decision boundary
Data needs	Performs well with small datasets	Performs better with large datasets
Best for	Explainability / visualization	Probability estimates / mathematical structure

Both can achieve similar accuracy; the choice depends on whether explainability or probability estimation matters more for the use case.

Key terms: root node, leaf node, Gini impurity, entropy, `max_depth`, overfitting, underfitting, `feature_importances_`.

7. Confusion Matrix Fundamentals

A confusion matrix is a table that shows classification performance in detail — revealing whether a model performs uniformly well or favors certain classes, which a single accuracy number cannot show.

Layout (Binary Classification)

- **Rows** represent the **actual** class labels.
- **Columns** represent the **predicted** class labels.
- **Diagonal cells** (top-left to bottom-right) represent correct classifications.
- **Off-diagonal cells** represent the two types of errors.

	Predicted Negative	Predicted Positive
Actual Negative	True Negative (TN)	False Positive (FP)
Actual Positive	False Negative (FN)	True Positive (TP)

- **True Positive (TP)**: actual positive, correctly predicted positive.
- **True Negative (TN)**: actual negative, correctly predicted negative.
- **False Positive (FP)** — Type I Error: actual negative, incorrectly predicted positive.
- **False Negative (FN)** — Type II Error: actual positive, incorrectly predicted negative.

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
```

```
cm = confusion_matrix(actual_labels, predicted_labels, labels=[0, 1])
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Fail',
'Pass'])
disp.plot(cmap='YlOrRd', colorbar=True)
```

Worked Example: 10 Learners (Pass/Fail)

Actual Predicted	Fail	Pass
Fail	3	1
Pass	1	5

This gives **TN = 3, FP = 1, FN = 1, TP = 5**, so:

Accuracy = (TP + TN) / Total = (5 + 3) / 10 = 0.80 — **80%**.

Multi-class confusion matrices follow the same layout principle, expanded to accommodate additional classes.

Key terms: confusion matrix, true positive, true negative, false positive, false negative, Type I error, Type II error.

8. Classification Metrics: Accuracy, Precision, Recall, F1 Score

Accuracy

Accuracy = $(TP + TN) / (TP + TN + FP + FN)$ — the proportion of correct predictions out of all instances.

- **Limitation:** accuracy can be highly misleading on **imbalanced datasets**. A model that blindly predicts the majority class for every instance can still achieve high accuracy while completely failing to detect the minority class. Always report a confusion matrix alongside accuracy to reveal this kind of failure.

Precision

Precision = $TP / (TP + FP)$ — of all instances predicted positive, what fraction were actually positive?

- High precision matters when **false positives are costly** — e.g. defect detection, medical diagnosis for serious diseases, fraud detection.
- On an imbalanced dataset, precision can reveal a serious weakness that accuracy hides entirely (e.g. very high accuracy alongside very low precision).

Worked Example: Factory Quality Control (500 Items)

A factory quality-control scenario with 500 products (485 good, 15 actually defective) produced this confusion matrix:

- **TP = 12** (defects correctly flagged), **FP = 28** (good items wrongly flagged as defective), **TN = 457** (good items correctly passed), **FN = 3** (defects missed).

Accuracy = $(12 + 457) / 500 = 0.938$ — **93.8%**, which looks strong.

Precision = $12 / (12 + 28) = 0.30$ — only **30%** — revealing that most items flagged as defective were actually good. The high accuracy alone completely hid this weakness, since it's dominated by the large number of correctly-passed good items.

Recall (Sensitivity)

Recall = $TP / (TP + FN)$ — of all instances that are actually positive, what fraction did the model correctly identify?

- High recall matters when **false negatives are costly** — e.g. catching all fraudulent transactions, or all patients with a serious disease.
- Precision focuses on **predicted positives** (denominator TP+FP); recall focuses on **actual positives** (denominator TP+FN) — this is the key structural difference between them.

The Precision–Recall Trade-off (via Threshold)

- **Raising the threshold** (e.g. 0.5 → 0.8): fewer instances predicted positive → fewer false positives → precision increases (more conservative positive predictions).

- **Lowering the threshold** (e.g. 0.5 $\rightarrow$ 0.3): more instances predicted positive $\rightarrow$ fewer false negatives $\rightarrow$ recall increases, but more false positives appear (more false alarms).
- The direction of this trade-off is a **general tendency**, not a per-dataset guarantee — whether a specific metric actually shifts on a specific split depends on where each sample's predicted probability falls relative to the two thresholds being compared. Always verify against your own computed confusion matrices rather than assuming the textbook direction must hold exactly for every dataset.

F1 Score

$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$ — the **harmonic mean** of precision and recall.

- The harmonic mean (rather than a simple average) is used because it penalizes extreme imbalance between precision and recall — if either metric is very low, F1 drops sharply, and F1 leans toward whichever of the two is lower.
- A single balanced metric, useful for comparing models without having to weigh precision and recall separately.

Worked Example: Forest-Fire Risk Detection (25 Regions)

Predicting whether a forest region is at fire risk (positive class) from sensor data (temperature, humidity, wind level):

Actual \ Predicted	No Fire Risk	Fire Risk
No Fire Risk	13 (TN)	2 (FP)
Fire Risk	4 (FN)	6 (TP)

- $\text{Precision} = 6 / (6 + 2) = 0.75$ — 75% of predicted fire-risk cases are genuinely fire-risk.
- $\text{Recall} = 6 / (6 + 4) = 0.60$ — the model catches only 60% of actual fire-risk cases, missing 40% of them — concerning for a safety application.
- $F1 = 2 \times (0.75 \times 0.60) / (0.75 + 0.60) \approx 0.667$ — closer to recall than precision, correctly flagging recall as the weaker, limiting metric here.

Comparing three model profiles on this same task:

Model	Precision	Recall	F1	Profile
A	0.90	0.53	0.67	Only raises alarms when very confident; misses many actual fire cases

Model	Precision	Recall	F1	Profile
B	0.40	0.95	0.57	Catches almost every fire case but triggers many false alarms, wasting response resources
C	0.70	0.78	0.74	Neither overly conservative nor overly sensitive — highest F1, best overall balance

This demonstrates why F1 penalizes imbalance: Model B's very high recall does not save it from the worst F1 of the three, because its precision is so low; Model C, despite having neither the highest precision nor the highest recall individually, wins overall because it balances both.

When to Prioritize Each Metric

- **Accuracy:** when classes are balanced and all errors have similar cost.
- **Precision:** when false positives are costly and high confidence in positive predictions is required.
- **Recall:** when false negatives are costly and capturing every positive case matters, even at the cost of some false alarms.
- **F1 / balanced approach:** when precision and recall matter roughly equally, or when a single summary number is needed for comparison.

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
```

```
accuracy_score(y_true, y_pred)
precision_score(y_true, y_pred)
recall_score(y_true, y_pred)
f1_score(y_true, y_pred)
```

Key terms: accuracy, precision, recall (sensitivity), F1 score, harmonic mean, threshold, precision-recall trade-off.

9. Choosing Between Regression and Classification Models

- Regression and classification are evaluated with entirely different metric families (RMSE/ R^2 vs. Accuracy/Precision/Recall/F1) that reflect fundamentally different problem semantics; they cannot be numerically compared to each other or averaged together to produce a meaningful combined score.
- Always compare models of the **same type**, trained/tested on the **same data**, for a fair evaluation.
- Both regression and classification comparisons use `train_test_split` — for classification, always use `stratify=y` to ensure proportional class representation in both the training and test sets (prevents a scenario where one class ends up disproportionately represented in only one split).
- **Feature scaling** with `StandardScaler` normalizes numeric features to mean 0, standard deviation 1. This benefits **scale-sensitive algorithms** — iterative, gradient-based learners like Logistic Regression and other distance-based methods — by speeding up their optimization and putting features on comparable scales for the optimizer. It does **not** apply universally: decision-tree splits are invariant to any monotonic rescaling of a feature (a tree picks the same threshold on raw or scaled data), and scaling a feature does not guarantee it will contribute more, or equally, to what the model actually learns — a feature's learned importance/weight still depends on its real relationship with the target, not just its scale. Fit the scaler on training data only, then transform both training and test data with those training-derived statistics (never fit on test data — that would leak information).

```
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
stratify=y)
```

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

- **Learning-based algorithms** (e.g. Linear Regression, Logistic Regression) fit weight parameters mathematically to the data — ordinary Linear Regression uses a closed-form least-squares solution, while Logistic Regression genuinely requires iterative optimization (no closed form exists for its loss function). Performance generally improves with more training data.
- **Tree-based algorithms** (e.g. Decision Trees) apply conditional (if-else) rules instead of fitting weights, and tend to work well even with smaller datasets; relative computational cost between the two families depends on dataset size, dimensionality, and tree depth — there is no fixed universal ranking.

- **DecisionTreeRegressor** (from `sklearn.tree`) is the regression counterpart to `DecisionTreeClassifier` — same rule-based splitting approach, but each leaf predicts a continuous value (typically the average target value of the training rows that reached it) instead of a class label.

Worked Example: Regression Model Comparison — Diabetes Dataset

Comparing Linear Regression against `DecisionTreeRegressor` on a diabetes dataset (442 patient records, 10 measurements predicting a disease-progression score):

Model	RMSE	R ²
Linear Regression	≈ 53	≈ 0.478
<code>DecisionTreeRegressor</code>	≈ 60	≈ 0.324

On this particular dataset, Linear Regression scored the lower RMSE and higher R². This is one dataset's result, not a universal ranking between the two algorithm families — the tree's own hyperparameters (e.g. `max_depth`) affect its score as much as its algorithm family does, so a fair comparison always holds hyperparameters and data splits constant before drawing a general conclusion.

Worked Example: Classification Model Comparison — Breast Cancer

Dataset

Comparing Logistic Regression against `DecisionTreeClassifier` on a breast-cancer dataset (569 diagnostic cases; malignant = positive class):

Model	Accuracy	Precision	Recall	F1
Logistic Regression	0.97	0.98	0.93	0.96
<code>DecisionTreeClassifier</code>	0.90	0.96	0.71	0.86

Both models look similar on accuracy, but **recall** reveals the real gap: the decision tree misses 29% of malignant cases (Recall 0.71) versus logistic regression's 7% miss rate (Recall 0.93) — a difference accuracy alone understates, and one that matters in medical diagnosis, where a missed malignant case (false negative) is far costlier than a false alarm. As with the Diabetes example above, this reflects one dataset and configuration, not a universal ranking between the two algorithm families.

Worked Example: Delivery Time — Regression vs. Classification Framing

- **As regression** ("how many minutes will this delivery take?"): for actual delivery times of [32, 45, 28, 58, 50, 38] minutes against predicted times of [30, 48, 32, 50, 40, 42] minutes, the residuals are [2, -3, -4, 8, 10, -4], giving **RMSE ≈ 5.90** and **R² ≈ 0.674** — a reasonably strong fit, with about two-thirds of the variance in delivery time explained by the model.
- **As classification** ("will this delivery be late or on time?"): a separate 20-delivery confusion matrix (5 TP, 10 TN, 2 FP, 3 FN) gives **Accuracy 0.75, Precision 0.71, Recall 0.625, F1 0.667**.

- Because these two framings measure fundamentally different things, the regression metrics (RMSE/ R^2) and the classification metrics (Accuracy/Precision/Recall/F1) from the two versions of the same delivery problem are never combined or averaged — each is meaningful only within its own problem type.

Key terms: stratify, StandardScaler, closed-form solution, iterative optimization, learning-based algorithm, tree-based algorithm, DecisionTreeRegressor.

10. Feature Selection & ML Pipelines

Pipelines

The Pipeline module automates a sequence of preprocessing and modeling steps (scaling, feature selection, cleaning, encoding, model training) in a single, reusable, reproducible object.

```
from sklearn.pipeline import Pipeline

pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('selector', SelectKBest(score_func=f_classif, k=3)),
    ('classifier', DecisionTreeClassifier())
])

pipeline.fit(X_train, y_train)
pipeline.predict(X_test)
```

- A single call to `.fit()` on the pipeline runs every step, in the order they are defined, on the training data — scaling statistics (mean, standard deviation) are learned only from the training data and then applied to the test data, preventing data leakage.
- Benefits: reproducibility, readable/organized code, prevents accidental leakage (e.g. scaling test data with test-set statistics), and makes revisiting old code months later much easier.

Redundant vs Irrelevant Features

- **Redundant features:** convey similar/overlapping information as an existing feature (e.g. age and a bucketed age_category). Identified via high pairwise **correlation** (near 1.0). Redundant features mainly add computational overhead and can destabilize coefficient interpretation in linear models (via multicollinearity), without necessarily hurting raw predictive accuracy much.
- **Irrelevant features:** do not meaningfully contribute to predicting the target (e.g. school_standard when predicting sport preference, once age/gender/height are known). Identified via statistical tests like ANOVA (F-test) or Chi-Square. Irrelevant features more directly degrade predictive accuracy by misleading the model with noise. The harm profile of redundant vs. irrelevant features differs by which criterion you care about — raw predictive accuracy vs. coefficient stability/interpretability.

Feature Selection with SelectKBest

SelectKBest with the `f_classif` scoring function computes an **ANOVA F-statistic** measuring how strongly each feature relates to the target class, ranks all features by that score, and keeps only the top `k`. Because noise/irrelevant features carry little real relationship with the target, they receive low scores and get dropped — this can substantially improve downstream metrics (e.g. F1 score) by removing noise that would otherwise mislead the model. Choosing `k` has no fixed formula — it is set through **experimentation** (trying a few values and comparing downstream metrics) or **domain knowledge** about how many features are genuinely likely to matter.

Fit the selector on the **training split only**, exactly like a scaler, so the score for each feature never sees test-set labels:

```
from sklearn.feature_selection import SelectKBest, f_classif

selector = SelectKBest(score_func=f_classif, k=3)
selector.fit(X_train, y_train)          # learn feature scores from training
data only
X_train_selected = selector.transform(X_train)
X_test_selected  = selector.transform(X_test)
```

Worked example — customer churn: a churn dataset built from 3 genuine features (`tenure_months`, `monthly_charges`, `contract_month_to_month`) plus 6 noise features (randomly generated, contributing no real predictive signal) showed the noise features collectively consuming **41% of the decision tree's importance score** when trained on all 9 features, dragging the F1 score down to **0.462**. After using SelectKBest to keep only the top 3 features, F1 improved dramatically to **0.824** — a concrete illustration of how irrelevant/noise features can actively mislead a model, and how removing them (rather than just adding more features) improves performance.

Preprocessing vs. Pipeline

- **Preprocessing** is the set of individual data-improvement operations themselves — cleaning, scaling, filling missing values, feature selection, encoding.
- **Pipeline** is the organizational framework that sequences those preprocessing operations together with the model into one reusable object.
- In other words, preprocessing steps are the components; the pipeline is the structure that chains them together in a fixed, reproducible order.

Feature Importance from Decision Trees

Decision trees compute a `feature_importances_` attribute after training, based on the **normalized reduction in whichever impurity criterion the tree was configured with** — Gini impurity by default, or entropy if the tree was trained with `criterion='entropy'`. It shows which features played the largest role in the classification decisions — a horizontal bar chart is a common way to visualize relative importance.

Key terms: Pipeline, redundant features, irrelevant features, correlation, ANOVA F-statistic, SelectKBest, feature_importances_.

11. Related Algorithms

These algorithms are useful to recognize by name alongside the models covered in depth above:

- **Support Vector Machine (SVM):** a learning-based classification algorithm, alongside Logistic Regression.
- **Neural Networks:** a more complex classifier capable of non-linear decision boundaries beyond what logistic regression can achieve.
- **Random Forest:** a tree-based algorithm, alongside Decision Trees.

Key terms: SVM, neural networks, random forest (recognition-level only).

Coverage checklist

This guide covers the full breadth of the material: ML problem types (supervised/unsupervised, regression/classification), simple and multiple linear regression, all four regression evaluation metrics (MAE/MSE/RMSE/R²), logistic regression and the sigmoid function, decision trees (theory, Gini/Entropy, overfitting, max_depth), the confusion matrix and all four classification metrics (Accuracy/Precision/Recall/F1), model-selection practices (stratified splits, feature scaling, learning-based vs. tree-based comparison), and feature engineering (redundant vs. irrelevant features, SelectKBest, pipelines, feature importance).